4.6 난수와 의사난수

- 실세계의 관측을 통해서 데이터를 획득하려면 **많은 비용이 필요**하며 **장시간의 관측**이 필요한 경우가 많음
- 따라서 실세계와 유사한 특성을 가지는 가상의 데이터를 만들어서 분석하고 시뮬레이션을 한 후에 이 모델을 실세계 데이터 분석에 적용
- 이를 위해서는 난수에 대한 개념을 이해하고 난수를 생성하는 알고리즘을 알아야 함
- 난수란 **인간의 의도와 무관하게 생성된 수**를 의미하므로, 인간이 만든 기계의 특정한 알고리즘을 통해서 난수를 생성한다는 것은 모순적으로 보임
- 하지만 난수와 매우 유사한 특징을 가지는 수를 생성하는 것은 가능한데 이러한 난수를 **의사 난수**pseudo random라고 함

4.6 난수와 의사난수

- 의사 난수는 엄밀한 의미에서의 난수가 아니지만, **난수와 유사한 기능을 함**

- 의사 난수 생성기의 특징은 이 생성기가 만들어낸 임의의 난수 값 X_n을 다음 난수 값 X_{n+1}을 만드는 재료로 사용한다는 점에 있음

$$X_{n+1} = (aX_n + b) \bmod m$$

- 위와 같은 예시에서 X는 의사 난수값의 **열**sequence이 되며, X_n값은 X_{n+1}값을 만드는 데 사용됨

- a 값과 b, m은 임의의 값으로 선택

- m은 (a * x + b)로 생성된 임의의 수를 모듈로 연산을 통하여 0에서 m-1 사이의 값으로 한정하는 기능을 함

4.6 난수와 의사난수

```python
def pseudo_rand(x):
    a = 1103515245
    b = 12345
    m = 2 ** 31
    new_x = (a * x + b) % m
    return new_x

seed = 100                # 초기값을 임의로 설정하자
x = pseudo_rand(seed)     # seed를 입력으로 하여 새로운 x를 만들자
print(x)
x = pseudo_rand(x)        # x_n을 입력으로 하여 새로운 x_(n+1)을 만들자
print(x)
x = pseudo_rand(x)        # x_n을 입력으로 하여 새로운 x_(n+1)을 만들자
print(x)
```

```
829870797
1533044610
1478614675
```

4.6 난수와 의사난수

- 난수를 생성하는 코드에는 다음 난수를 생성하기 위한 최초 값이 필요한데 이를 **시드값**seed number이라고 함

- 위의 코드에서는 100이라는 임의의 수로 주었으나 보통 시드값은 컴퓨터로부터 시간 정보를 얻어서 이 값을 사용하는 경우가 많음

- 만약 위의 코드에서 **시드값**을 101로 변경하면 1933386042와 같은 전혀 다른 난수를 얻게 되며 이 후에 생성되는 난수들도 달라지게 됨

```
# 101을 입력으로 하여 새로운 x을 만들자
x = pseudo_rand(101)
print(x)
```

```
1933386042
```

4.6 난수와 의사난수

- 난수를 생성하는 함수나 모듈은 대부분의 프로그래밍 언어에서 라이브러리 형태로 제공
- 넘파이의 경우 random이라는 서브 모듈 아래에 난수를 생성하는 다양한 함수들이 제공됨
- 0에서 1 사이의 난수 다섯 개를 생성하는 넘파이 명령을 살펴보자.

```
# 0에서 1사이의 난수 다섯 개를 생성
np.random.rand(5)
```

```
array([0.8202121 , 0.28819938, 0.52844401, 0.99236168, 0.04999261])
```

```
np.random.rand(2, 3)  # 형상이 (2, 3)인 난수로 이루어진 다차원 배열을 반환함
```

```
array([[0.76680396, 0.00239144, 0.26799324],
       [0.41502864, 0.62034193, 0.05864043]])
```

- rand() 함수는 인자로 2, 3과 같은 튜플값을 받음으로써 난수 배열의 형상을 지정할 수도 있음

4.6 난수와 의사난수

- 넘파이로 가상의 성인 10명의 키(cm) 데이터를 생성하는 경우를 고려해 보자.
- 데이터는 150에서 190 사이의 임의의 값을 가진다고 가정
- 정수값을 생성하는 경우 넘파이의 random 서브 모듈에 있는 randint() 함수를 사용
- randint() 함수는 인자로 최소값, 최대값, 원소의 개수를 줄 수 있으며, 최대값이 191일 경우 191은 원소에 포함되지 않음

```
# 키가 150 센티미터에서 190 센티미터 사이인 성인의 데이터
np.random.randint(150, 191, size=10)
```

```
array([186, 157, 167, 153, 159, 173, 176, 187, 163, 179])
```

4.6 난수와 의사난수

- 위의 난수 집합은 무작위적으로 키가 150에서 190 사이의 성인 데이터를 생성

- 데이터의 범위를 130에서 220까지로 늘려 보자.

```
# 키가 130 센티미터에서 220 센티미터 사이인 성인의 데이터
np.random.randint(130, 221, size=10)
```

```
array([151, 182, 131, 217, 159, 167, 131, 193, 189, 150])
```

- 위의 코드가 만들어 내는 값에는 131이 두 개나 포함되어 있음

- 성인 10명 중에서 키가 131cm인 성인이 두 명이나 포함되어 있는 것은 다소 비현실적

- 이를 어떻게 해결해야 할까?

4.7 난수와 정규분포

- 앞 절에서 살펴본 바와 같이 데이터 과학에서 **난수**random number는 매우 중요한 역할을 함

- 난수의 중요성은 다음과 같이 여러 측면에서 볼 수 있음

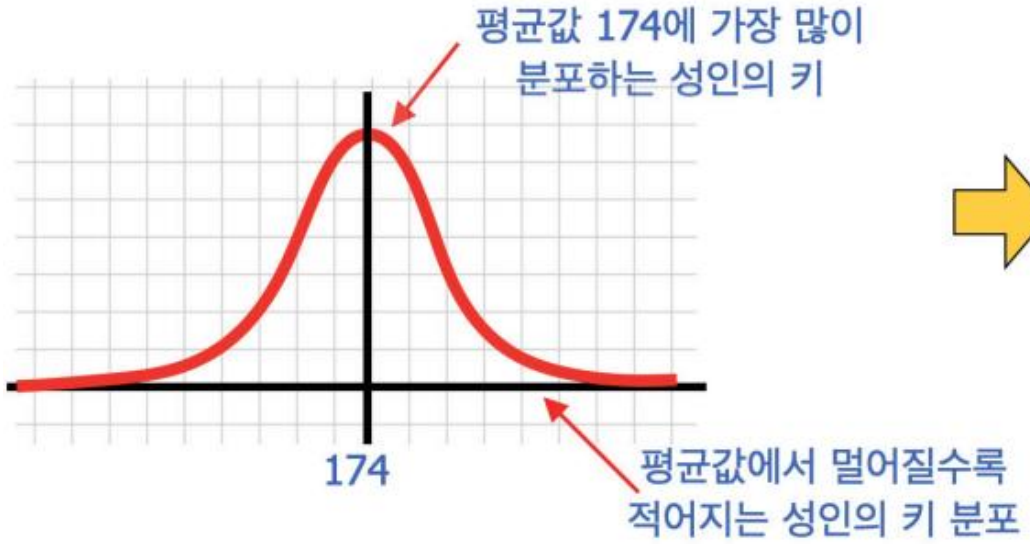
1. **실험 및 시뮬레이션:** 데이터 과학에서 난수는 다양한 실험적 시나리오와 시뮬레이션을 생성하는 데 필수적이다. 예를 들어, 몬테카를로 시뮬레이션과 같은 기법에서 난수는 확률적인 사건의 결과를 모델링하는 데 사용된다. 이러한 접근법은 금융, 물리학, 엔지니어링 등 다양한 분야에서 복잡한 시스템의 동작을 이해하고 예측하는 데 중요하다.
2. **알고리즘 테스트 및 검증:** 데이터 과학 알고리즘과 모델을 테스트하고 검증할 때 난수를 사용하여 다양한 데이터 패턴을 생성하고, 이를 통해 알고리즘의 강건성과 일반화 능력을 평가한다.
3. **데이터 샘플링:** 난수는 데이터 세트에서 무작위 샘플을 추출하는 데 사용된다. 이는 통계적 추론과 모델링에서 중요한 역할을 하며, 데이터의 편향을 줄이고, 더 일반화 가능한 결론을 도출하는 데 기여한다.
4. **머신러닝과 딥러닝:** 많은 머신러닝 알고리즘, 특히 딥러닝에서는 난수를 사용하여 초기 가중치를 설정한다. 이 초기화 과정은 모델 학습의 성능과 안정성에 중요한 영향을 미친다.

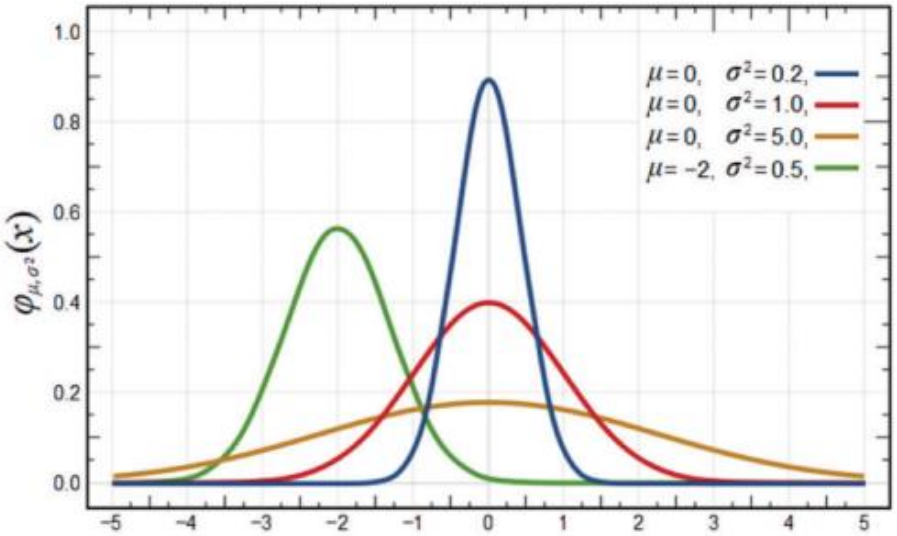
4.7 난수와 정규분포

- 데이터 분석을 위해 많은 데이터가 필요할 경우 이것을 직접 수집하는 경우도 있으나 많은 비용과 시간이 소요됨
- 예를 들어 대한민국 성인 남성의 키와 몸무게, 발의 길이를 모으는 경우 우리나라 성인 남성이 대략 1,500만명가량 된다고 가정하면 이들의 데이터를 직접 모으는 것은 힘든 작업이 될 것임
- 이 경우 난수를 사용하면 실제 데이터에 거의 가까운 데이터를 쉽게 생성할 수 있음

4.7 난수와 정규분포

- 조사에 의하면 우리나라 성인 남성의 키 평균은 약 174cm로 알려져 있음
- 이들의 분포는 174에 가장 밀집해 있을 것이며 174의 근방인 173과 175와 같은 값에도 많이 분포
- 그러나 153이나 195와 같은 값에는 보다 적게 분포
- 이러한 분포는 다음 그림과 같이 **종모양의 대칭적인 분포**가 되며, 이 분포를 **정규분포**normal distribution 라고 함





평균과 표준편차에 따른 다양한 정규분포

4.7 난수와 정규분포

- 넘파이에서 제공하는 정규분포 함수는 randn()을 사용

- 예를 들어서 정규분포를 따르는 난수 4개를 생성하는 코드는 다음과 같음

```
np.random.seed(42)   # 난수를 만들기 위한 초기값
np.random.randn(4)   # 4 개의 난수를 생성한다. 이 난수는 정규분포를 따름
```

```
array([0.37454012, 0.95071431, 0.73199394, 0.59865848])
```

- 이것은 인자로 넘어온 개수만큼의 난수를 발생시키는데, 이 난수들은 평균이 0이고 표준편차가 1인 **표준 정규분포**standard normal distribution를 따르게 됨

4.7 난수와 정규분포

- 만약 평균이 10인 정규분포를 따르는 난수를 만들기 위해서는 이 값에 10을 더하면 됨

```
10 + np.random.randn(4)  # 평균이 10인 4 개의 난수를 생성한다. 이 난수는 정규분포를 따름
```

```
array([ 9.76584663,  9.76586304, 11.57921282, 10.76743473])
```

- 정규분포의 표준편차를 3으로 두려면 이 난수 값에 3을 곱하면 됨

```
3 * np.random.randn(4)  # 표준편차가 3인 4 개의 난수를 생성한다. 이 난수는 정규분포를 따름
```

```
array([-3.03849336,  0.942742  , -2.72407223, -4.2369111 ])
```

4.7 난수와 정규분포

- 평균값을 mu라는 변수로 두고, 표준편차를 sigma로 설정하여 이에 맞는 정규분포를 생성하고 싶다면 다음과 같은 코드를 사용

```
mu = 10
sigma = 3
rand_nums = mu + sigma * np.random.randn(4)
rand_nums
```

```
array([14.39694631,  9.3226711 , 10.20258461,  5.72575544])
```

4.7 난수와 정규분포

도전문제 4.5 정규분포를 따르는 난수 생성

난이도 중

평균이 174이고 표준편차가 12인 정규분포를 따르는 난수를 생성하여라. 이 난수를 (4, 3) 형상의 다차원 배열에 저장하고 그 값을 출력하여라.

```
평균이 174이고 표준편차가 12인 난수들

array([[178.3396323 , 166.25856294, 178.33674727],
       [192.4564388 , 173.57008753, 192.77572387],
       [142.56305875, 183.86283005, 175.04456482],
       [170.41191179, 175.10112932, 150.14917302]])
```

4.8 다양한 난수 생성 기능

- randint(a, b)는 정수 a와 정수 b 사이의 난수를 생성하여 반환하는 함수
- 이때 정수 a는 생성될 난수의 범위에 포함되지만 정수 b는 포함되지 않으며 b-1이 이 범위에 포함됨
- 따라서 1에서 100 사이의 임의의 정수를 생성하고자 한다면 다음과 같은 코드를 입력하여야 함

```
np.random.randint(1, 101)
```

```
83
```

- 또한 size라는 키워드 인자를 통해서 생성할 난수의 수를 지정할 수 있음
- 예를 들어서 주사위 던지기 5번을 연속으로 하여 나오는 수를 시뮬레이션한다면 다음과 같이 코드를 작성할 수 있음

```
np.random.randint(1, 7, size=5)  # 1에서 6사이의 난수 5개를 생성
```

```
array([3, 3, 5, 4, 3])
```

4.8 다양한 난수 생성 기능

- 만일 이 수들을 모두 12개 생성하여 형상이 (2, 6)인 이차원배열의 원소로 채우려면 다음과 같이 입력

```
np.random.randint(1, 7, size=(2, 6))
```

```
array([[4, 5, 1, 4, 2, 6],
       [5, 4, 1, 1, 3, 3]])
```

4.8 다양한 난수 생성 기능

- random 서브 모듈에 포함된 중요한 난수 관련 함수들

함수 이름	설명
rand()	균등분포의 난수 표본을 추출한다.
randn()	표준정규분포의 난수 표본을 추출한다.
randint()	최소값과 최대값을 인자로 받아, 주어진 범위 내의 정수 난수를 생성한다. 이때 최대값은 포함하지 않는다.
normal()	평균과 표준편차를 인자로 받아, 정규분포의 난수 표본을 추출한다.
uniform()	균등 분포에서 표본을 추출한다.
shuffle()	주어진 리스트나 다차원 배열의 순서를 임의로 섞는다.
permutation()	주어진 범위의 수를 생성하여 섞거나, 배열을 임의의 순서로 섞는다.

4.8 다양한 난수 생성 기능

- 이 함수들을 사용하여 임의의 수를 생성하거나 섞을 수 있음

```
a = np.arange(10)        # 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 값을 생성
np.random.shuffle(a)
a                         # a에는 순서가 뒤섞인 값이 들어있다
```

```
array([2, 8, 9, 6, 4, 0, 1, 5, 7, 3])
```

```
# 배열을 임의의 순서로 섞는다
b = np.random.permutation([2, 4, 6, 8, 10])
b
```

```
array([ 8,  4, 10,  6,  2])
```

```
# 0에서 9까지의 연속된 정수 10개를 생성하여 이를 임의의 순서로 섞는다
c = np.random.permutation(10)
c
```

```
array([8, 5, 1, 7, 0, 2, 4, 9, 6, 3])
```

4.8 다양한 난수 생성 기능

- permutation()과 shuffle()은 유사해 보이지만 결정적인 차이가 존재

- permutation()의 경우 인자로 들어오는 다차원 배열을 복사해서 섞어주지만 shuffle()은 다차원 배열을 섞는 일을 **인플레이스**inplace로 함

- **인플레이스란 새로운 배열을 만드는 것이 아니라 배열 자체의 원소값을 변화시키는 기능**

4.8 다양한 난수 생성 기능

- 이 기능을 다음 코드로 확인해 보자.

```
a = np.array([1, 2, 3, 4, 5])
b = np.random.permutation(a)  # a 배열 원소의 값을 섞어서 b 배열을 만든다
c = np.array([1, 2, 3, 4, 5])
np.random.shuffle(c)          # c 배열 원소의 값을 섞어준다(인플레이스)
a
```

```
array([1, 2, 3, 4, 5])
```

```
b
```

```
array([1, 3, 4, 5, 2])
```

```
c
```

```
array([1, 5, 3, 2, 4])
```

4.8 다양한 난수 생성 기능

도전문제 4.6 데이터 섞어서 나누기

난이도 중

arange()를 사용하여 1에서 50까지의 원소를 가지는 다차원 배열 x를 만들자. 이 배열의 원소 50개를 랜덤하게 섞은 후 80%의 데이터는 train_data에 넣고 나머지 20%의 데이터는 test_data라는 배열에 넣어서 이 두 배열을 반환하는 함수 train_test_split()을 만들자. 다음과 같이 반환된 배열 값을 각각 출력하여라.

```
train_data(80%) =
[24 17 48 35  8 20 50 25 33 49 46 43 18 28 34 16 37 40 12  6 21 31 10  2
 22 19 44  7 45 36 29 11 41 23 30  4 38 32 13 47]
test_data(20%) =
[42 15  9 27 39 14 26  5  1  3]
```

LAB4-5 로또 번호를 생성하자

실습 시간

세계 여러 나라에서 시행중인 복권 시스템 중 하나인 로또는 1에서 45까지 임의의 정수를 6개 맞히는 규칙을 가지고 있다. 로또 시행사는 매주 6개의 수와 추가의 보너스 번호를 발표하고 로또 참가자는 자신이 가진 6개의 수를 비교하여 순위가 결정된다. 당연히 6개를 모두 맞추면 1등이 되고, 5개를 맞추면 2등이 되며, 4개를 맞추면 3등이 되는 등수 결정 규칙을 가지고 있다.

여러분이 직접 임의의 6개의 수를 고르는 것이 아니라 넘파이의 random 서브 모듈을 활용하여 다음과 같이 5개의 로또 번호를 자동으로 생성하여 출력하는 프로그램을 만들어보자.

원하는 결과

```
이번 주의 추천 로또번호 : [2, 5, 7, 9, 25, 39]
```

힌트

넘파이를 이용하여 1에서 45까지의 연속적인 값을 가지는 배열을 만들기 위해서는 다음과 같이 arange()를 사용한다.

```
a = np.arange(1, 46)
print(a)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45]
```

a 다차원 배열의 원소를 random 서브모듈의 shuffle() 함수를 이용하여 섞은 다음, 이 배열 원소들 중에서 가장 앞에 있는 5개의 수를 슬라이싱을 통하여 추출하도록 하자.

```
np.random.shuffle(a)
print(a[:6])
```

LAB4-5 로또 번호를 생성하자

해답 코드

```
import numpy as np

a = np.arange(1, 46)
np.random.shuffle(a)
print('이번 주의 추천 로또번호 :',a[:6])
```

도전문제 4.7 로또 번호 출력하기

난이도 중

이번에는 매주 로또 시행사에서 발표하는 당첨 번호를 출력해 보자. 임의의 로또 당첨 번호와 함께 다음과 같이 보너스 번호를 큰 괄호로 묶어서 출력해 보자.

```
이번 주의 당첨 로또번호 + 보너스
[11, 12, 20, 32, 37, 42] + [9]
```

LAB4-6 평균과 중앙값 계산 연습

실습 시간

프로 팀에 소속된 농구 선수들 200명의 데이터를 정규분포를 이용하여 자동으로 생성해 보자. 200명의 선수가 각각 키, 몸무게를 가지므로 배열의 형태는 (2, 200)가 될 것이다. 이 배열의 첫 행은 키, 두 번째 행은 몸무게 데이터가 입력될 것이다. 데이터는 정규분포를 가지는 randn() 함수를 이용하여 생성할 것이다. 이 때 키는 194 센티미터, 몸무게는 78 킬로그램을 평균으로 하고, 표준편차는 각각 11과 8이 되도록 하자. 이렇게 생성된 데이터에 대하여 평균값과 중앙값을 구하여 다음과 같이 출력하도록 하자.

원하는 결과

```
200명의 농구 선수들에 대한 정보
신장의 평균값: 194.2183101958153
신장의 중앙값: 194.1064453046699
체중의 평균값: 77.52762372474302
체중의 중앙값: 78.17552578638133
```

힌트

배열을 0으로 채워서 만들 때는 넘파이의 zeros() 함수를 이용하면 된다. 다음과 같은 방법으로 (2, 200) 형상의 players 배열을 생성하도록 하자.

```
num_player = 200
players = np.zeros((2, num_player))
```

평균값과 표준편차에 따라 정규분포를 따르는 난수 생성법은 이미 학습했다. 선수 정보 배열의 가장 첫 행을 신장으로 할 경우 다음과 같은 방법으로 평균이 194이고 표준편차가 11인 200개의 원소를 생성할 수 있다.

```
players[0] = 11 * np.random.randn(num_player) + 194
```

LAB4-6 평균과 중앙값 계산 연습

해답 코드

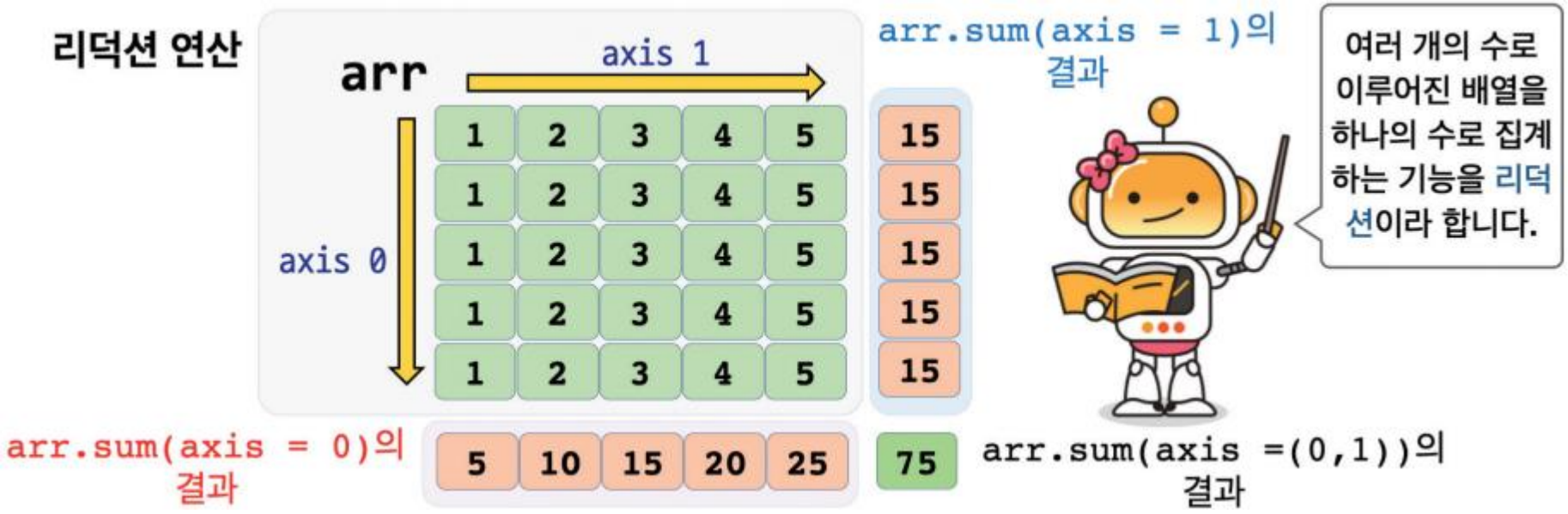
```
num_player = 200
players = np.zeros((2, num_player))
players[0] = 11 * np.random.randn(num_player) + 194
players[1] = 8 * np.random.randn(num_player) + 78

print(f'{num_player}명의 농구 선수들에 대한 정보')
heights = players[0]
print('신장의 평균값:', np.mean(heights))
print('신장의 중앙값:', np.median(heights))

weights = players[1]
print('체중의 평균값:', np.mean(weights))
print('체중의 중앙값:', np.median(weights))
```

4.9 리덕션: 강력한 배열 연산

- 넘파이의 다차원 배열은 **리덕션**reduction이라는 이름의 기능을 제공함
- 리덕션이라는 것은 여러 개의 수로 이루어진 배열을 하나의 수로 **집계**aggregation하는 기능
- 넘파이는 이 집계를 위한 함수 로 sum(), mean(), min(), max() 같은 함수를 제공함
- 이 함수들은 대상이 되는 다차원 배열의 원소에 대해 각각 총합(sum), 평균(mean), 최소값(min), 최대값(max)을 구하는 일을 하며, 사용자가 임의의 집계 함수를 직접 만들 수도 있음

4.9 리덕션: 강력한 배열 연산

- 넘파이의 리덕션은 적용되는 축을 지정하여 이루어짐
- 다음과 같이 2차원 배열에 sum()을 적용할 때 axis=0 혹은 axis=1을 적용하면 그림과 같이 해당 축 방향으로 리덕션을 수행
- axis=(0, 1)과 같이 축을 두 개 모두 주면 전체 리덕션이 이루어짐



4.9 리덕션: 강력한 배열 연산

- 코드를 통해 이 연산을 수행해 보자.

```
arr = np.array([[1,2,3,4,5],
                [1,2,3,4,5],
                [1,2,3,4,5],
                [1,2,3,4,5],
                [1,2,3,4,5]])
print('np.sum(arr, axis = 0)의 결과')
print(np.sum(arr, axis = 0))
print('np.sum(arr, axis = 1)의 결과')
print(np.sum(arr, axis = 1))
print('np.sum(arr, axis = (0, 1))의 결과')
print(np.sum(arr, axis= (0, 1)))  # axis = (0, 1)은 생략 가능
```

```
np.sum(arr, axis = 0)의 결과
[ 5 10 15 20 25]
np.sum(arr, axis = 1)의 결과
[15 15 15 15 15]
np.sum(arr, axis = (0, 1))의 결과
75
```

4.9 리덕션: 강력한 배열 연산

- np.sum(arr, axis = 0) 기능은 arr.sum(axis = 0)으로 대체 가능함

```
print(arr.sum(axis = 0))          # np.sum(arr, axis = 0)와 동일한 결과
print(arr.sum(axis = 1))
print(arr.sum(axis = (0, 1)))
```

```
[ 5 10 15 20 25]
[15 15 15 15 15]
75
```

- **리덕션 연산**은 **벡터화 연산을 통해 이루어지므로 높은 성능**을 보임

- 큰 규모의 배열을 만들고 각 원소의 합을 원소별 접근을 통해서 계산하는 방법과 리덕션 기법을 이용한 방법으로 수행해 보고, 소요 시간을 비교해 보자.

4.9 리덕션: 강력한 배열 연산

- 성능 비교를 위하여 다음과 같이 1000x1000 크기의 다차원 배열 arr을 생성하자.

```
import numpy as np
import time

n = 1000
arr = np.random.rand(n, n)
print(f'arr 다차원 배열의 원소 개수: {arr.size}')
```

```
arr 다차원 배열의 원소 개수: 1000000
```

4.9 리덕션: 강력한 배열 연산

- 모든 원소에 접근해 그 값을 변수 sum_naive에 누적하는 방식으로 배열 원소의 합을 구하는 코드의 결과와 소요 시간을 출력해 보자.

```
sum_naive = 0          # for 반복문을 사용하여 원소별 접근을 하는 경우
start = time.time()

for i in range(n):
    for j in range(n):
        sum_naive += arr[i,j]
end = time.time()
print(n, 'x', n, "배열의 원소 합", sum_naive)
print("계산 시간: ", end-start)
```

```
1000 x 1000 배열의 원소 합 500048.24990126066
계산 시간:  0.39148378372192383
```

4.9 리덕션: 강력한 배열 연산

- 이제 이 값을 벡터화 연산이 적용되는 리덕션으로 구해보자.
- 원소의 합은 동일한 값이 출력되고, 소요 시간은 0.0013초로 **300배 이상 빨라진 것**을 확인할 수 있음

```
start = time.time()        # 반복문 대신 리덕션을 사용하는 경우
# 앞서 수행한 연산과 동일한 연산을 np.sum()을 통해서 연산을 수행함
sum_reduction = np.sum(arr, axis = (0, 1))
end = time.time()
print(n, 'x', n, "배열의 원소 합", sum_naive)
print("계산 시간: ", end-start)
```

```
1000 x 1000 배열의 원소 합 500048.2499012628
계산 시간:  0.00128173828125
```

- 넘파이는 **벡터화 연산, 그리고 리덕션을 통해 대규모 수치 배열에 대한 빠른 계산 기능을 제공**
- 이를 잘 활용하기 위해서는 원소별 접근, 행이나 열 단위 접근을 최소한으로 사용하는 것이 좋음

4.10 상관관계 계산하기

- 일반적으로 키가 큰 사람의 몸무게가 키가 작은 사람에 비해서 많이 나가는 것을 볼 수 있음
- 혹은 집이 부유한 사람일수록 비싼 차를 타는 경향이 있는 것도 볼 수 있음
- 이러한 관측으로부터, 키와 몸무게, 연간 소득과 보유한 자동차의 가격은 상호 의존성이 있는 관계에 있다고 볼 수 있으며, 이러한 **의존성의 정도**를 **상관관계**correlation라고 함
- **상관계수**correlation coefficient란 이 **상관관계를 -1에서 1 사이의 수치값으로 나타낸 것**

4.10 상관관계 계산하기

- corrcoef(x, y) 함수는 요소들의 상관계수를 계산함
- 여기서 x와 y는 데이터를 담고 있는 리스트 또는 배열
- 결과는 다음과 같은 상관계수행렬을 반환함 (C_{xy}는 x와 y의 상관계수)

$$\begin{bmatrix} C_{xx} & C_{xy} \\ C_{yx} & C_{yy} \end{bmatrix}$$

- 완전한 음의 상관관계일 경우 상관계수는 -1, 상관관계가 전혀 없을 때 상관계수는 0, 완전한 양의 상관관계를 가질 때 상관계수가 1이 됨
- 자기 자신과의 상관계수는 항상 1이므로 이 행렬의 대각원소들은 모두 1이 됨
- 또한, x와 y의 상관계수와 y와 x의 상관계수는 같기 때문에 이 행렬은 대칭행렬이 됨

4.10 상관관계 계산하기

- 예를 들어 x는 1에서 99까지의 정수를 가지고, y는 1부터 99까지 정수의 제곱을 갖는다고 하고 실제 상관계수를 계산해 보자.

```
x = [ 1, 2, 3, 4, ..., 97, 98, 99 ]
y = [ 1, 4, 9, 16, ..., 9409, 9604, 9801]
```

- 이 경우 x의 값이 증가하면 y의 값도 증가하므로 둘 사이의 상관관계는 매우 높게 계산될 것이다.

```
x = np.arange(100)        # 0에서 99까지의 값을 요소로 하는 다차원 배열
y = x ** 2                # 0에서 99까지의 값의 제곱을 요소로 하는 다차원 배열

result = np.corrcoef(x, y)
print(result)
```

```
[[1.         0.96764439]
 [0.96764439 1.        ]]
```

4.10 상관관계 계산하기

- 만일 x 값과 y 값 사이에 y = 2x와 같은 상관관계가 있을 경우 x가 1만큼 증가할 경우 y는 2만큼 증가하는 특징이 있음
- 이러한 관계를 가지는 함수를 **선형 함수**linear function라고 함
- 선형 함수에 대한 상관계수를 확인해 보자.

```
x = np.arange(100)        # 0에서 99까지의 값을 요소로 하는 다차원 배열
y = 2 * x                 # 0에서 99까지의 값에 대한 선형관계를 가지는 다차원 배열
result = np.corrcoef(x, y)
print(result)
```

```
[[1. 1.]
 [1. 1.]]
```

- 이와 같이 선형 함수 관계에 있는 두 변수들은 완벽한 의존관계가 있기 때문에 상관계수는 1이 됨

4.10 상관관계 계산하기

- 3개 이상의 변수를 사용하여 상관계수를 계산할 수도 있음

- 다음과 같은 3개의 데이터의 상관계수를 계산해 보자.

```
x = np.arange(100)      # 0에서 99까지의 값을 요소로 하는 다차원 배열
y = x ** 2              # 0에서 99까지의 값에 대하여 x**2 값을 가지는 다차원 배열
z = np.sin(3.14 * x/100)
```

- 리스트 z는 주기가 200인 **사인**sine 함수로 0도에서 180도까지 그려짐

- 사인함수는 주기함수이므로 선형, 비선형적으로 증가하는 앞의 두 리스트 x, y와는 큰 상관관계가 없을 것으로 예측할 수 있음

4.10 상관관계 계산하기

- 다음과 같은 행렬을 얻기 위해서는 리스트 x, y, z를 모두 corrcoef() 함수에 넘겨주면 됨

$$\begin{bmatrix} C_{xx} & C_{xy} & C_{xz} \\ C_{yx} & C_{yy} & C_{yz} \\ C_{zx} & C_{zy} & C_{zz} \end{bmatrix}$$

- corrcoef(x, y, z)와 같이 구현할 경우 오류가 발생하므로 아래와 같이 리스트나 튜플로 묶어서 입력

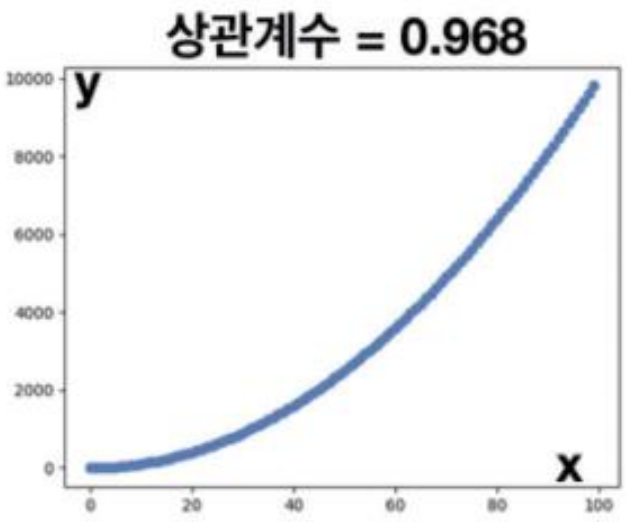
```
x = np.arange(100)      # 0에서 99까지의 값을 요소로 하는 다차원 배열
y = x ** 2              # 0에서 99까지의 값에 대하여 x**2 값을 가지는 다차원 배열
z = np.sin(3.14 * x/100)
result = np.corrcoef([x, y, z])  # 또는 np.corrcoef((x, y, z))
print(result)
```

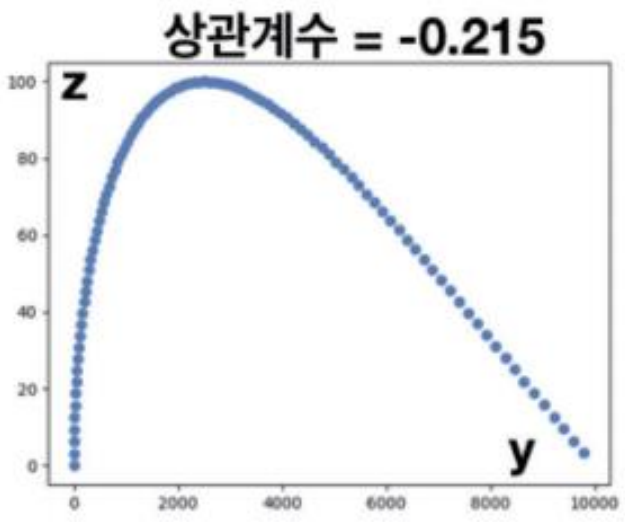
```
[[ 1.          0.96764439  0.03763255]
 [ 0.96764439  1.         -0.21532645]
 [ 0.03763255 -0.21532645  1.        ]]
```

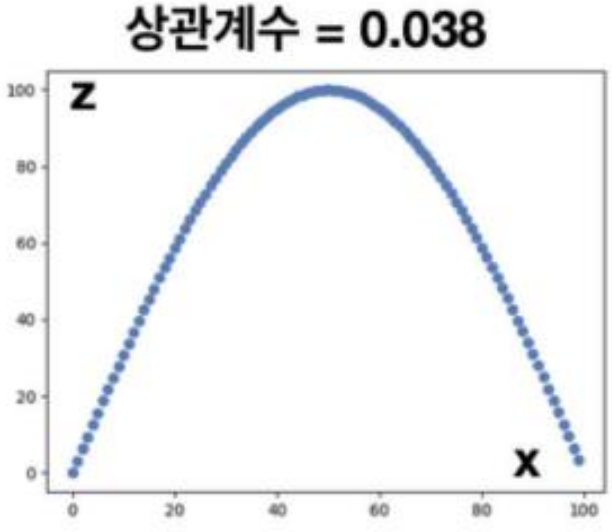
4.10 상관관계 계산하기

- 결과를 해석하면 x와 y의 상관관계는 0.968, y와 z의 상관관계는 -0.215, z와 x의 상관관계는 0.038 임을 알 수 있음

- x와 y의 관계, y와 z의 관계, z와 x의 관계를 각각 그림으로 나타내면 다음과 같음







핵심정리

- 넘파이는 다차원 배열의 최대값, 최소값, 평균값을 구하는 기능과 정렬 기능을 제공한다.
- 평균과 같이 데이터의 전체적인 경향을 나타내는 정보를 대표값이라고 한다.
- 실제 데이터들 중에서 비정상적으로 큰 값이나 작은 값을 지칭하는 용어가 바로 이상치이다.
- 분산을 이용하여 평균값으로부터 데이터가 퍼진 정도를 구할 수 있다.
- 넘파이는 의사 난수를 생성할 수 있는 다양한 방법을 제공한다.
- 난수는 실험 및 시뮬레이션과 데이터 샘플링, 머신러닝과 딥러닝 알고리즘에서도 중요하다.
- 다차원 배열은 리덕션이라는 강력한 집계 기능을 제공한다.

핵심정리

데이터의 전체적인 경향이란 데이터 집합이 보여주는 일반적인 움직임이나 패턴을 말하지요.
평균과 같은 것이 데이터의 경향을 나타내는 대표적인 값이지요.
평균값은 어떠한 문제점이 있을까요?

평균값은 전체 데이터 값의 합을 구한 후 이를 데이터의 수로 나눈 것이죠.
하지만 데이터에서 특정 값이 너무 크거나 작은 경우 평균값으로 전체의 일반적인 패턴을 표현하기에 부적절 한 것 같습니다.

그래요. 그것이 평균값이 가진 한계에요.
이러한 이유로 중앙값이나 최빈값을 사용한답니다.

핵심정리

그렇군요.
난수는 데이터 과학에서 왜 중요하게 다루나요?

난수는 예측가능한 패턴이 없는 수를 말하지요.
이러한 이유로 데이터 과학을 위한 시뮬레이션, 알고리즘 테스트와 검증에 사용될 수 있답니다.

Questions?